GAME PROGRAMMING COURSE — UNITY

データをセーブしよう

PlayerPrefs と JSON ファイルで、ゲームを閉じてもデータを残す

対象：高等部2年（全2コマ）

SAVE
DATA

LESSON 1

PlayerPrefs で保存する

目標：前回の GameDataHolder を、ゲームを閉じてもデータが残るようにする

導入

セーブはどこで使う？

ハイスコア

前回までの最高記録を覚えておき、タイトルやリザルト画面に表示する。

設定

音量・画面の明るさ・ボタン配置など、プレイヤーが決めた設定。

進み具合

クリアしたステージ、持っているアイテムやお金。「つづきから」に必要。

POINT 前回の GameDataHolder は、シーンをまたいで値を持てるが**ゲームを閉じると消える**。閉じても残すには「セーブ」が必要。

考え方

実行中のデータと、保存したデータ

メモリ

変数の値・GameDataHolder
ゲームを閉じると消える

セーブ →

← ロード

ストレージ

PC やスマホに書き込んだファイル
ゲームを閉じても残る

保存のしかた	向いているもの	この授業
PlayerPrefs	ハイスコア・音量など、少しの値	LESSON 1
JSON ファイル	進み具合など、まとまったデータ	LESSON 2

基本

PlayerPrefs の使い方

書き方	意味
PlayerPrefs.SetInt(キー, 値)	int を保存する（SetFloat・SetString も同じ）
PlayerPrefs.GetInt(キー, 初期値)	読み込む。保存がなければ初期値が返る
PlayerPrefs.HasKey(キー)	そのキーで保存されているか
PlayerPrefs.DeleteKey(キー)	そのキーのデータを消す
PlayerPrefs.DeleteAll()	全部消す（使うときは注意）
PlayerPrefs.Save()	すぐにストレージへ書き込む

POINT　Save を呼ばなくてもゲーム終了時に書き込まれるが、**強制終了すると消える**ことがある。大事な値は Save まで書く。

基本

保存できるのは3種類だけ

int	float	string
整数	**小数**	**文字列**
スコア・ステージ番号・コイン	音量・プレイ時間	プレイヤー名・選んだキャラ

C# bool を保存したいとき

```
// bool は保存できないので、1 と 0 に変えて保存する
PlayerPrefs.SetInt("TutorialDone", tutorialDone ? 1 : 0);

// 読むときは 1 と比べて bool に戻す
bool tutorialDone = PlayerPrefs.GetInt("TutorialDone", 0) == 1;
```

基本

PlayerPrefs はどこに保存される？

環境	保存される場所
Windows（ビルド）	レジストリ：HKCU\Software\[会社名]\[製品名]
Unity エディター	レジストリ：HKCU\Software\Unity\UnityEditor\[会社名]\[製品名]
Mac	~/Library/Preferences/com.[会社名].[製品名].plist
WebGL	ブラウザの IndexedDB

POINT 会社名・製品名は **Project Settings** > **Player** の Company Name / Product Name。

⚠ テスト中に全部消したいときは、メニューの **Edit** > **Clear All PlayerPrefs**。

前回のスクリプト

前回の GameDataHolder 全文（1/2）

C# GameDataHolder.cs（前回）

```
using UnityEngine;

public class GameDataHolder : PersistentSingleton<GameDataHolder>
{
    [SerializeField] public int score;

    protected override void Awake()
    {
        base.Awake();

        score = PlayerPrefs.GetInt("score");
    }

    protected override void OnDestroy()
    {
        base.OnDestroy();

        PlayerPrefs.SetInt("score",score);
    }
```

前回の GameDataHolder 全文（2/2）

C# GameDataHolder.cs（前回）（続き）

```
21      public void Load()
22      {
23
24      }
25
26      public void Save()
27      {
28
29      }
30
31      /// <summary>
32      /// ゲーム起動時に生成
33      /// </summary>
34      [RuntimeInitializeOnLoadMethod]
35      private static void Init()
36      {
37          GameDataHolder pref = Resources.Load<GameDataHolder>("Singletons/GameDataHolder");
38          if (pref ≠ null) Instantiate(pref);
39      }
40  }
```

前回のスクリプト

このままだと困ること

① Load と Save が空のまま

読み込みは Awake、保存は OnDestroy に直接書いてある。セーブしたいときに呼べない。

② キーが文字列の手打ち

"score" を2か所で打っている。1文字ちがうだけで別のデータになる。

③ OnDestroy で保存している

2つ目が作られて消されたときにも呼ばれる。「閉じたとき」なら OnApplicationQuit の方がはっきりする。

④ PlayerPrefs.Save() がない

強制終了したときに、保存されないことがある。

読み込み・保存は Load / Save にまとめて、呼ぶタイミングを決める

完成したスクリプト

PlayerPrefs 版 GameDataHolder 全文（1/3）

C# GameDataHolder.cs（PlayerPrefs 版）

```
using UnityEngine;

public class GameDataHolder : PersistentSingleton<GameDataHolder>
{
    // PlayerPrefs のキー（打ち間違えないように定数にする）
    const string SCORE_KEY = "Score";
    const string HIGH_SCORE_KEY = "HighScore";

    [SerializeField] public int score;
    [SerializeField] public int highScore;

    protected override void Awake()
    {
        base.Awake();
        if (Instance ≠ this) return;    // 2つ目なら何もしない

        Load();
    }

    // ゲームを閉じるときに自動でセーブ
    protected override void OnApplicationQuit()
    {
        Save();
```

PlayerPrefs 版 GameDataHolder 全文（2/3）

C# GameDataHolder.cs（PlayerPrefs 版）（続き）

```
24          base.OnApplicationQuit();
25      }
26
27      /// <summary>
28      /// PlayerPrefs から読み込む
29      /// </summary>
30      public void Load()
31      {
32          score = PlayerPrefs.GetInt(SCORE_KEY, 0);
33          highScore = PlayerPrefs.GetInt(HIGH_SCORE_KEY, 0);
34      }
35
36      /// <summary>
37      /// PlayerPrefs に書き込む
38      /// </summary>
39      public void Save()
40      {
41          PlayerPrefs.SetInt(SCORE_KEY, score);
42          PlayerPrefs.SetInt(HIGH_SCORE_KEY, highScore);
43          PlayerPrefs.Save();
44      }
45
46      /// <summary>
```

完成したスクリプト

PlayerPrefs 版 GameDataHolder 全文（3/3）

C# GameDataHolder.cs（PlayerPrefs 版）（続き）

```
47        /// 今のスコアがハイスコアを超えていたら更新してセーブする
48        /// </summary>
49        public void UpdateHighScore()
50        {
51            if (score > highScore)
52            {
53                highScore = score;
54                Save();
55            }
56        }
57
58        /// <summary>
59        /// ゲーム起動時に生成
60        /// </summary>
61        [RuntimeInitializeOnLoadMethod]
62        private static void Init()
63        {
64            GameDataHolder pref = Resources.Load<GameDataHolder>("Singletons/GameDataHolder");
65            if (pref == null) Instantiate(pref);
66        }
67    }
```

サンプルスクリプトの解説

PlayerPrefs 版 ① キーと変数

C# GameDataHolder.cs（PlayerPrefs 版）

```
1  using UnityEngine;
2
3  public class GameDataHolder : PersistentSingleton<GameDataHolder>
4  {
5      // PlayerPrefs のキー（打ち間違えないように定数にする）
6      const string SCORE_KEY = "Score";
7      const string HIGH_SCORE_KEY = "HighScore";
8
9      [SerializeField] public int score;
10     [SerializeField] public int highScore;
```

const string SCORE_KEY

キーは定数にする

打ち間違いがあればコンパイルエラーで気づける。

int highScore

ハイスコアを追加

スコアと一緒にセーブする。

サンプルスクリプトの解説

PlayerPrefs 版 ② 起動時と終了時

C# GameDataHolder.cs（続き）

```
12      protected override void Awake()
13      {
14          base.Awake();
15          if (Instance ≠ this) return;    // 2つ目なら何もしない
16
17          Load();
18      }
19
20      // ゲームを閉じるときに自動でセーブ
21      protected override void OnApplicationQuit()
22      {
23          Save();
24          base.OnApplicationQuit();
25      }
```

Instance ≠ this

2つ目なら Load しない。カメラ演出の回のルールと同じ。

Load()

Awake では Load を呼ぶだけにする。

OnApplicationQuit

ゲームを閉じるときに1回だけ呼ばれる。ここで Save。

base.OnApplicationQuit()

基底クラスの処理も忘れずに呼ぶ。

サンプルスクリプトの解説

PlayerPrefs 版 ③ Load と Save

C# GameDataHolder.cs（続き）

```
27      /// <summary>
28      /// PlayerPrefs から読み込む
29      /// </summary>
30      public void Load()
31      {
32          score = PlayerPrefs.GetInt(SCORE_KEY, 0);
33          highScore = PlayerPrefs.GetInt(HIGH_SCORE_KEY, 0);
34      }
35
36      /// <summary>
37      /// PlayerPrefs に書き込む
38      /// </summary>
39      public void Save()
40      {
41          PlayerPrefs.SetInt(SCORE_KEY, score);
42          PlayerPrefs.SetInt(HIGH_SCORE_KEY, highScore);
43          PlayerPrefs.Save();
44      }
```

GetInt(KEY, 0)

保存がないとき（初めて遊ぶとき）は 0。

SetInt(KEY, 値)

保存する項目の数だけ書く。

PlayerPrefs.Save()

すぐにストレージへ書き込む。

PlayerPrefs 版 ④ ハイスコアの更新

C# GameDataHolder.cs（続き）

```
46      /// <summary>
47      /// 今のスコアがハイスコアを超えていたら更新してセーブする
48      /// </summary>
49      public void UpdateHighScore()
50      {
51          if (score > highScore)
52          {
53              highScore = score;
54              Save();
55          }
56      }
```

POINT ハイスコアを超えたときだけ書き換えて、**すぐに Save**。58 行目からの Init は前回のまま変えていない。

まとめて確認

前回からの変更点

	前回	PlayerPrefs 版
読み込み	Awake に直接書く	Awake から Load() を呼ぶ
保存	OnDestroy に直接書く	OnApplicationQuit から Save() を呼ぶ
キー	"score" を手打ち	SCORE_KEY などの定数
保存するもの	score だけ	score と highScore
書き込み	PlayerPrefs.Save() なし	Save() の最後で PlayerPrefs.Save()
2つ目の対策	なし	Instance != this なら何もしない

使い方

どこからでも呼べる

C# 使う側（ゲームオーバーやタイトルのスクリプト）

```
// ゲームオーバーになったとき
GameDataHolder.Instance.UpdateHighScore();

// タイトル画面でハイスコアを表示する
highScoreText.text = $"BEST {GameDataHolder.Instance.highScore}";

// ステージクリアなど、区切りのいいところで
GameDataHolder.Instance.Save();
```

POINT LoadとSaveをpublicにしたので、**好きなタイミング**でセーブできる。

準備

Unity で確かめる

手順	作業	確認すること
1	GameDataHolder.cs を PlayerPrefs 版に書きかえる	プレハブは前回のまま使う
2	再生して、前回の Demo で↓キーを何回か押す	スコアが増える
3	再生を止めて、もう一度再生する	スコアが前の値から始まる
4	再生中に Hierarchy の DontDestroyOnLoad を開く	GameDataHolder の Score が見える
5	Edit ＞ Clear All PlayerPrefs を押してから再生	スコアが 0 に戻る

POINT エディターで再生を止めたときも **OnApplicationQuit** が呼ばれるので、手順3で確かめられる。

考え方

PlayerPrefs の苦手なこと

項目が増えると大変

コインやステージを足すたびに、キー・GetInt・SetInt がそれぞれ増える。

まとめて保存できない

配列・List・自分で作ったクラスは、そのまま保存できない。

中身が見えにくい

レジストリに入るので、何が保存されているか確かめにくい。

まとまったデータは、JSON ファイルにして保存する（LESSON 2）

演習

PlayerPrefs で保存しよう

1 **GameDataHolder を PlayerPrefs 版にする**

前回のスクリプトを書きかえ、ゲームを閉じてもスコアが残ることを確かめよう。

2 **ハイスコアを表示する**

ゲームオーバーで UpdateHighScore を呼び、タイトル画面に BEST を表示しよう。

3 **音量も保存する**

float の bgmVolume を GameDataHolder に足し、SetFloat / GetFloat で保存しよう。

ヒント：キーは `BGM_VOLUME_KEY` のように定数にする

4 **発展：記録をリセットするボタン**

タイトル画面にリセットボタンを作ろう。DeleteAll は使わず、DeleteKey で消す。

LESSON 2

JSON ファイルで保存する

目標：GameDataHolder を、まとまったデータを1つのファイルで保存する形に改良する

基本

JSON とは

JSON save.json

```json
{
    "score": 1200,
    "highScore": 4500
}
```

データを**文字で書き表すための決まった形**。
人間が読めて、プログラムでも扱いやすい。

{ }

ひとまとまりのデータ（オブジェクト）。

"名前": 値

名前と値のペア。カンマで区切って並べる。

[ ]

配列。値を順番に並べる。

考え方

JSON でセーブ・ロードする流れ

セーブ



ロード



POINT JsonUtility が**クラス** ⇔ **文字列**、File が**文字列** ⇔ **ファイル**を担当する。

基本

JsonUtility の使い方

書き方	意味
JsonUtility.ToJson(obj)	クラスを JSON の文字列にする（1行）
JsonUtility.ToJson(obj, true)	改行・字下げ付きで見やすくする
JsonUtility.FromJson<T>(json)	JSON の文字列から T 型のクラスを作る
JsonUtility.FromJsonOverwrite(json, obj)	今あるクラスに値を上書きする

POINT JsonUtility は Unity に最初から入っているので、追加のインストールなしで使える。

注意

JSON にできるもの・できないもの

できる

- [Serializable] を付けたクラス
- public の変数、[SerializeField] の変数
- int・float・bool・string
- 配列・List

できない（保存されない）

- [Serializable] がないクラス
- プロパティ（get / set）
- Dictionary
- 2次元配列（int[,]）

⚠ できないものは**エラーにならずに、こっそり保存されない。**セーブしたのに値が戻らないときは、ここを確認する。

基本

ファイルはどこに保存する？

Application.persistentDataPath ＝ ゲームが自由に書き込める専用のフォルダ。

環境	実際の場所
Windows	C:\Users\[ユーザー名]\AppData\LocalLow\[会社名]\[製品名]
Mac	~/Library/Application Support/[会社名]/[製品名]
Android	/storage/emulated/0/Android/data/[パッケージ名]/files

⚠ ビルドしたゲームには **Assets フォルダは存在しない**。Assets に保存してよいのはエディターで確認するときだけ（次のページ）。

基本

#if でエディターとビルド版を切り替える

C# #if の書き方

```
#if UNITY_EDITOR
    // Unity エディターで再生したときだけ、こちらが使われる
    static readonly string FOLDER_PATH = Path.Combine(Application.dataPath, "SaveData");
#else
    // ビルドしたゲームでは、こちらだけが入る
    static readonly string FOLDER_PATH = Application.persistentDataPath;
#endif
```

#if UNITY_EDITOR

エディターのときだけ

保存先を Assets/SaveData にして、Project ウィンドウで中身を見られるようにする。

#else

ビルド版のとき

Unity 推奨の persistentDataPath に保存する。

if 文とのちがい

コンパイルの時点で選ぶ

使われない側はビルドに入らない。実行中に切り替わることはない。

⚠ Assets/SaveData は自分のセーブデータ。Git を使うときは **.gitignore に追加**して、コミットしないようにする。

基本

ファイルを読み書きする

書き方	意味
using System.IO;	File・Path・Directory を使うために必要
Path.Combine(フォルダ, ファイル名)	OS に合った区切り文字でパスをつなぐ
Directory.CreateDirectory(フォルダ)	フォルダを作る（すでにあれば何もしない）
File.WriteAllText(パス, 文字列)	ファイルに書き込む（なければ作る）
File.ReadAllText(パス)	ファイルの中身を全部読む
File.Exists(パス)	ファイルがあるか
File.Delete(パス)	ファイルを消す

POINT パスは "/" で手でつながずに **Path.Combine** を使う。Windows と Mac で区切り文字がちがうため。

完成したスクリプト

SaveData 全文

C# SaveData.cs

```
using System;

// セーブする中身（GameDataHolder が持つ）
[Serializable]
public class SaveData
{
    public int score = 0;
    public int highScore = 0;
}
```

サンプルスクリプトの解説

SaveData セーブする中身

C# SaveData.cs

```
1  using System;
2
3  // セーブする中身（GameDataHolder が持つ）
4  [Serializable]
5  public class SaveData
6  {
7      public int score = 0;
8      public int highScore = 0;
9  }
```

[Serializable]

JSON にするために必要。付け忘れると何も保存されない。

score / highScore

PlayerPrefs 版で GameDataHolder が持っていた値を、ここに移す。

= 0

初めて遊ぶとき（新しいデータ）の値になる。

項目を増やすとき

ここに1行足すだけ。GameDataHolder の Load / Save は変えなくてよい。

完成したスクリプト

JSON 版 GameDataHolder 全文（1/5）

C# GameDataHolder.cs（JSON 版）

```
using System.IO;
using UnityEngine;

public class GameDataHolder : PersistentSingleton<GameDataHolder>
{
    const string FILE_NAME = "save.json";

#if UNITY_EDITOR
    // エディター：中身を確認しやすいように Assets/SaveData
    static readonly string FOLDER_PATH = Path.Combine(Application.dataPath, "SaveData");
#else
    // ビルド版：Unity 推奨の保存場所
    static readonly string FOLDER_PATH = Application.persistentDataPath;
#endif

    // セーブするデータをまとめて持つ
    public SaveData data = new SaveData();

    protected override void Awake()
    {
        base.Awake();
```

JSON 版 GameDataHolder 全文（2/5）

C# GameDataHolder.cs（JSON 版）（続き）

```
22          if (Instance ≠ this) return;    // 2つ目なら何もしない
23
24          Load();
25      }
26
27      // ゲームを閉じるときに自動でセーブ（PC 向け）
28      protected override void OnApplicationQuit()
29      {
30          Save();
31          base.OnApplicationQuit();
32      }
33
34      // スマホでホーム画面に戻ったときに自動でセーブ
35      void OnApplicationPause(bool isPaused)
36      {
37          if (isPaused) Save();
38      }
39
40      /// <summary>
41      /// JSON ファイルから読み込む（ファイルがなければ新しいデータ）
42      /// </summary>
```

JSON 版 GameDataHolder 全文（3/5）

C# GameDataHolder.cs（JSON 版）（続き）

```
43      public void Load()
44      {
45          string path = GetFilePath();
46          if (!File.Exists(path))
47          {
48              data = new SaveData();
49              return;
50          }
51
52          string json = File.ReadAllText(path);
53          data = JsonUtility.FromJson<SaveData>(json);
54      }
55
56      /// <summary>
57      /// JSON ファイルに書き込む
58      /// </summary>
59      public void Save()
60      {
61          string json = JsonUtility.ToJson(data, true);
62          File.WriteAllText(GetFilePath(), json);
63      }
```

JSON 版 GameDataHolder 全文（4/5）

C# GameDataHolder.cs（JSON 版）（続き）

```
64      /// <summary>
65      /// はじめから遊ぶ（ファイルを消して新しいデータにする）
66      /// </summary>
67      public void ResetData()
68      {
69          string path = GetFilePath();
70          if (File.Exists(path)) File.Delete(path);
71
72          data = new SaveData();
73      }
74
75      /// <summary>
76      /// 今のスコアがハイスコアを超えていたら更新してセーブする
77      /// </summary>
78      public void UpdateHighScore()
79      {
80          if (data.score > data.highScore)
81          {
82              data.highScore = data.score;
83              Save();
```

JSON 版 GameDataHolder 全文（5/5）

C# GameDataHolder.cs（JSON 版）（続き）

```
85              }
86          }
87
88          // セーブファイルの場所を返す（フォルダがなければ作る）
89          static string GetFilePath()
90          {
91              Directory.CreateDirectory(FOLDER_PATH);
92              return Path.Combine(FOLDER_PATH, FILE_NAME);
93          }
94
95          /// <summary>
96          /// ゲーム起動時に生成
97          /// </summary>
98          [RuntimeInitializeOnLoadMethod]
99          private static void Init()
100         {
101             GameDataHolder pref = Resources.Load<GameDataHolder>("Singletons/GameDataHolder");
102             if (pref != null) Instantiate(pref);
103         }
104     }
```

サンプルスクリプトの解説

JSON 版 ① ファイル名と保存するフォルダ

C# GameDataHolder.cs（JSON 版）

```
1  using System.IO;
2  using UnityEngine;
3
4  public class GameDataHolder : PersistentSingleton<GameDataHolder>
5  {
6      const string FILE_NAME = "save.json";
7
8  #if UNITY_EDITOR
9      // エディター：中身を確認しやすいように Assets/SaveData
10     static readonly string FOLDER_PATH = Path.Combine(Application.dataPath, "SaveData");
11 #else
12     // ビルド版：Unity 推奨の保存場所
13     static readonly string FOLDER_PATH = Application.persistentDataPath;
14 #endif
```

#if UNITY_EDITOR

変数を作るところで切り替え

エディターとビルド版で保存先が変わる。

static readonly

const にはできない

実行するまで値が決まらないので readonly。

JSON 版 ② データをまとめて持つ

C# GameDataHolder.cs（続き）

```
16      // セーブするデータをまとめて持つ
17      public SaveData data = new SaveData();
18
19      protected override void Awake()
20      {
21          base.Awake();
22          if (Instance ≠ this) return;    // 2つ目なら何もしない
23
24          Load();
25      }
```

SaveData data

score・highScore などを1つにまとめて持つ。

Awake

PlayerPrefs 版と同じ。2つ目でなければ Load。

サンプルスクリプトの解説

JSON 版 ③ 終了時の自動セーブ

C# GameDataHolder.cs（続き）

```
27      // ゲームを閉じるときに自動でセーブ（PC 向け）
28      protected override void OnApplicationQuit()
29      {
30          Save();
31          base.OnApplicationQuit();
32      }
33
34      // スマホでホーム画面に戻ったときに自動でセーブ
35      void OnApplicationPause(bool isPaused)
36      {
37          if (isPaused) Save();
38      }
```

`OnApplicationQuit`

PC でゲームを閉じたとき。

`OnApplicationPause(true)`

スマホで裏に回ったとき。スマホは終了時に呼ばれないことが多い。

サンプルスクリプトの解説

JSON 版 ④ Load

C# GameDataHolder.cs（続き）

```
40      /// <summary>
41      /// JSON ファイルから読み込む（ファイルがなければ新しいデータ）
42      /// </summary>
43      public void Load()
44      {
45          string path = GetFilePath();
46          if (!File.Exists(path))
47          {
48              data = new SaveData();
49              return;
50          }
51
52          string json = File.ReadAllText(path);
53          data = JsonUtility.FromJson<SaveData>(json);
54      }
```

`!File.Exists(path)`

ファイルがない＝初めて遊ぶとき。新しい SaveData にする。

`ReadAllText`

ファイルの中身を文字列で読む。

`FromJson<SaveData>`

文字列から SaveData を作り、data に入れる。

サンプルスクリプトの解説

JSON 版 ⑤ Save と ResetData

C# GameDataHolder.cs（続き）

```
56      /// <summary>
57      /// JSON ファイルに書き込む
58      /// </summary>
59      public void Save()
60      {
61          string json = JsonUtility.ToJson(data, true);
62          File.WriteAllText(GetFilePath(), json);
63      }
64
65      /// <summary>
66      /// はじめから遊ぶ（ファイルを消して新しいデータにする）
67      /// </summary>
68      public void ResetData()
69      {
70          string path = GetFilePath();
71          if (File.Exists(path)) File.Delete(path);
72
73          data = new SaveData();
74      }
```

`ToJson(data, true)`

data を見やすい JSON の文字列にする。

`WriteAllText`

前のファイルは上書きされる。

`ResetData()`

「はじめから」用。ファイルを消して新しいデータにする。

JSON 版 ⑥ ハイスコアとファイルの場所

C# GameDataHolder.cs（続き）

```
76      /// <summary>
77      /// 今のスコアがハイスコアを超えていたら更新してセーブする
78      /// </summary>
79      public void UpdateHighScore()
80      {
81          if (data.score > data.highScore)
82          {
83              data.highScore = data.score;
84              Save();
85          }
86      }
87
88      // セーブファイルの場所を返す（フォルダがなければ作る）
89      static string GetFilePath()
90      {
91          Directory.CreateDirectory(FOLDER_PATH);
92          return Path.Combine(FOLDER_PATH, FILE_NAME);
93      }
```

data.score

値は data の中にあるので、data. を付けて使う。

CreateDirectory

初めてのときは SaveData フォルダがないので作る。

Path.Combine

フォルダとファイル名をつなぐ。95 行目からの Init は前回のまま。

まとめて確認

PlayerPrefs 版からの変更点

	PlayerPrefs 版	JSON 版
値の持ち方	score・highScore を別々に持つ	SaveData data にまとめて持つ
保存先	PlayerPrefs（レジストリ）	save.json ファイル
読み込み	項目の数だけ GetInt	FromJson で1回
保存	項目の数だけ SetInt	ToJson で1回
項目を増やすとき	キー・GetInt・SetInt を足す	SaveData に1行足すだけ
使う側の書き方	Instance.score	Instance.data.score

⚠ 使う側の書き方が変わるので、前回の GameDataHolderDemo は **score を data.score** に書きかえる。

確認

保存されたファイルを見てみよう

JSON save.json（↓キーを3回押して再生を止めた後）

```json
{
    "score": 3,
    "highScore": 0
}
```

1 **Project ウィンドウを見る**

Assets/SaveData に save.json ができる。出ないときは右クリック ＞ Refresh。

2 **ダブルクリックで開く**

JSON の中身がエディターで読める。

3 **値を書き換える**

score の値を変えて保存し、もう一度再生してみる。

考え方

セーブはいつする？

タイミング	例	保存のしかた	注意
ステージクリア	ゴールした・ボスを倒した	Save() を呼ぶ	いちばん基本。結果が決まった瞬間
チェックポイント	中間地点を通った	Save() を呼ぶ	復帰する場所も一緒に保存する
大事な入手	レアアイテム・お金	Save() を呼ぶ	回数が多すぎるなら、まとめて保存
設定を変えた	オプション画面を閉じた	PlayerPrefs	スライダーを動かすたびには保存しない
ゲームを閉じる	ウィンドウの × ボタン	OnApplicationQuit	強制終了・スマホでは呼ばれないことがある
ホームに戻る	スマホでアプリを切り替えた	OnApplicationPause	スマホではここが「終了」の代わり

POINT ゲームを閉じたときの自動セーブは**保険**。大事なデータは、区切りのいいタイミングでセーブしておく。

注意

避けたいセーブのしかた

Update で毎フレーム

ファイルの書き込みは重い。1秒に60回書き込むと、ゲームがカクつく。

ピンチの途中で

残りHP 1・ボス戦の途中で保存すると、ロードしても負けるしかない「詰み」になる。

終了時だけ

落ちたり電源が切れたりすると、遊んだ分が全部消える。

「区切りのいいところでセーブ」＋「終了時の自動セーブ」の二段構え

考え方

ロードはいつする？

起動時に自動で読み込む（今回）

- セーブデータは1つだけ
- どのシーンから再生しても読み込み済み
- タイトルの「つづきから」は、読み込んだデータを使うだけ
- GameDataHolder の Awake で Load する

スロットを選んだときに読み込む

- セーブデータを3つなどから選ぶ
- 選ぶまでは、どのデータを読むか決まらない
- 選択画面では、一覧表示のために全スロットを確認する
- RPG などでよく見る形

POINT 今回は左の形。右の**セーブスロット式**は後で紹介する。

使い方

区切りのいいところでセーブする

C# 使う側（ゴールやタイトルのスクリプト）

```
// ゲームオーバーになったとき
GameDataHolder.Instance.UpdateHighScore();

// ステージクリアなど、区切りのいいところで
GameDataHolder.Instance.Save();

// タイトルの「はじめから」ボタン
GameDataHolder.Instance.ResetData();
SceneLoadManager.Instance.LoadScene("Stage1");
```

POINT 「つづきから」は、起動時にもう読み込んでいるので**シーンを読み込むだけ**。

紹介

セーブスロット式

スロット 1	スロット 2	スロット 3
スコア 1200 ハイスコア 4500	データなし	スコア 300 ハイスコア 800



POINT ファイルを **save_1.json・save_2.json・save_3.json** のように分けるだけで作れる。

スロット式にするときの変更点

C# GameDataHolder.cs（変更例）

```
// スロット式にするなら、GameDataHolder をこう変える
int currentSlot = 1;

// Awake では Load しない（スロットを選ぶまで読まない）

// スロット選択画面でボタンを押したとき
public void SelectSlot(int slot)
{
    currentSlot = slot;
    Load();
}

// ファイル名にスロット番号を入れる（static を外す）
string GetFilePath()
{
    Directory.CreateDirectory(FOLDER_PATH);
    return Path.Combine(FOLDER_PATH, $"save_{currentSlot}.json");
}
```

currentSlot

今遊んでいるスロットの番号を覚えておく。

SelectSlot(slot)

ロードのタイミングが「起動時」から「選んだとき」に変わる。

save_{currentSlot}

Load も Save も、今のスロットのファイルを使う。

比較

PlayerPrefs と JSON ファイル

	PlayerPrefs	JSON ファイル
保存できるもの	int・float・string	クラスまるごと（配列・List も）
保存先	レジストリなど	ファイル（エディターは Assets/SaveData）
中身の確認	しにくい	エディターやメモ帳で読める
手軽さ	1行で保存できる	クラスと保存処理を作る
向いているもの	音量などの設定	進み具合・アイテム・セーブスロット

POINT どちらか1つではなく、**設定は PlayerPrefs、進み具合は JSON** のように使い分けることが多い。

注意

セーブデータで気をつけること

書き換えられる

JSON はメモ帳で書き換えられる。オンラインのランキングなどには使わない。

壊れることがある

書き込み中に電源が切れると、ファイルが途中で終わってしまう。

読み込みに失敗したとき（発展）

C# Load の中を try-catch で囲む

```
try
{
    string json = File.ReadAllText(path);
    data = JsonUtility.FromJson<SaveData>(json);
}
catch (Exception e)
{
    Debug.LogWarning($"セーブデータを読めませんでした: {e
    data = new SaveData();
}
```